

Lab Experiment

Experiment 3 : Water Jug Problem using Python

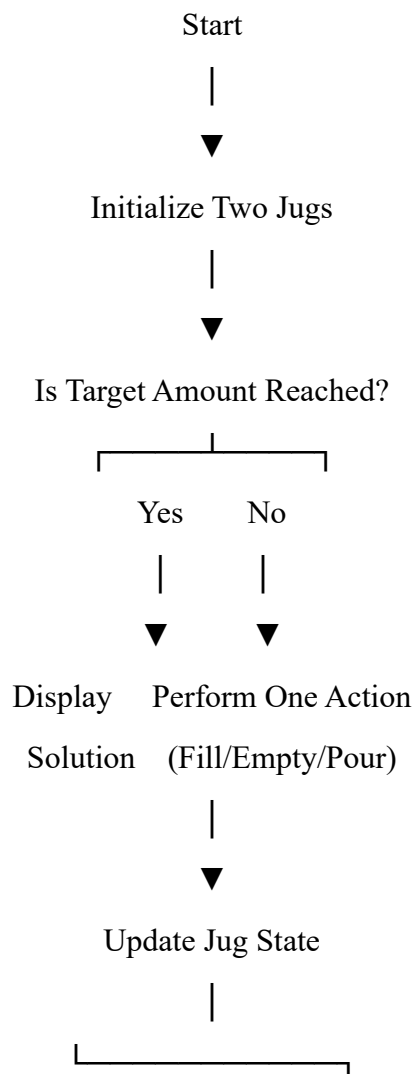
Aim

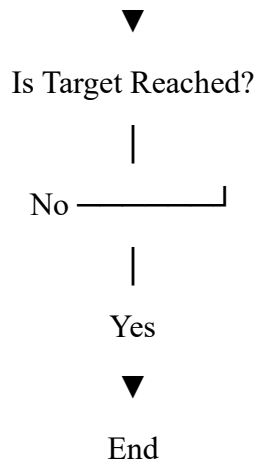
To solve the Water Jug Problem using Python by finding a sequence of steps to measure the required amount of water.

Objective

- To measure a required amount of water using two jugs of different capacities.
- Use operations such as **fill**, **empty**, and **pour** between the jugs.
- Find the solution using a **State Space Search** algorithm (BFS/DFS).

Flowchart:





Algorithm

1. Start with both jugs empty.
2. Fill the 4-liter jug.
3. Pour water into the 3-liter jug.
4. Empty the 3-liter jug when it becomes full.
5. Transfer the remaining water.
6. Repeat until 2 liters remain in the 4-liter jug.
7. Display each step.

Pseudo Code

START

Initialize Jug1 = 0 and Jug2 = 0

Fill Jug1

Pour Jug1 into Jug2

Empty Jug2

Pour remaining water from Jug1 to Jug2

Fill Jug1

Pour Jug1 into Jug2 until Jug2 is full

Print each step

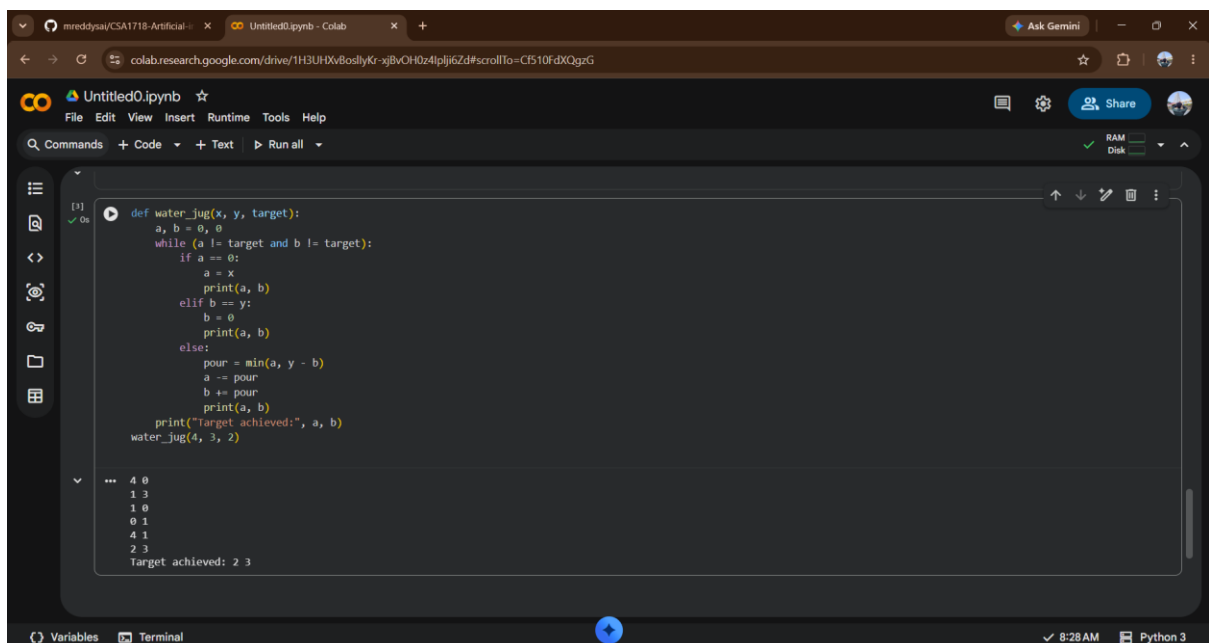
STOP

Python Code

```
def water_jug(x, y, target):  
    a, b = 0, 0  
  
    while (a != target and b != target):  
        if a == 0:  
            a = x  
            print(a, b)  
        elif b == y:  
            b = 0  
            print(a, b)  
        else:  
            pour = min(a, y - b)  
            a -= pour  
            b += pour  
            print(a, b)  
  
    print("Target achieved:", a, b)  
  
water_jug(4, 3, 2)
```

Output

Water Jug Problem Solution



The screenshot shows a Google Colab notebook titled 'Untitled0.ipynb'. The code cell contains the same Python code as shown in the previous block. The output of the code is displayed below the code cell, showing the sequence of states (a, b) as the water is poured between the two jugs until the target amount of 2 is achieved in one of the jugs.

```
[1] def water_jug(x, y, target):  
    a, b = 0, 0  
    while (a != target and b != target):  
        if a == 0:  
            a = x  
            print(a, b)  
        elif b == y:  
            b = 0  
            print(a, b)  
        else:  
            pour = min(a, y - b)  
            a -= pour  
            b += pour  
            print(a, b)  
    print("Target achieved:", a, b)  
    water_jug(4, 3, 2)  
  
... 4 0  
1 3  
1 0  
0 3  
4 1  
2 3  
Target achieved: 2 3
```

Goal Achieved: 2 liters in the 4L jug.

Result

The Water Jug Problem was successfully solved using Python. The required **2 liters** of water was obtained in the **4-liter jug** by following the sequence of operations.

Experiment 4 : Cryptarithmic Problem using Python

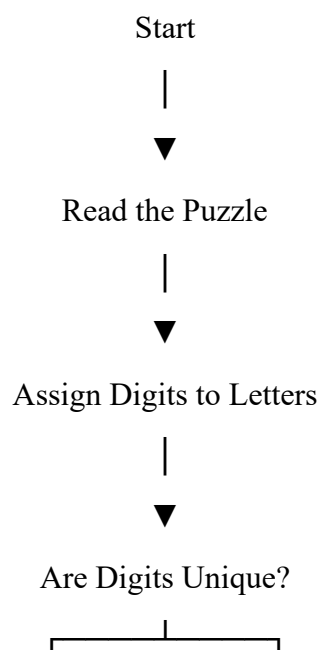
Aim

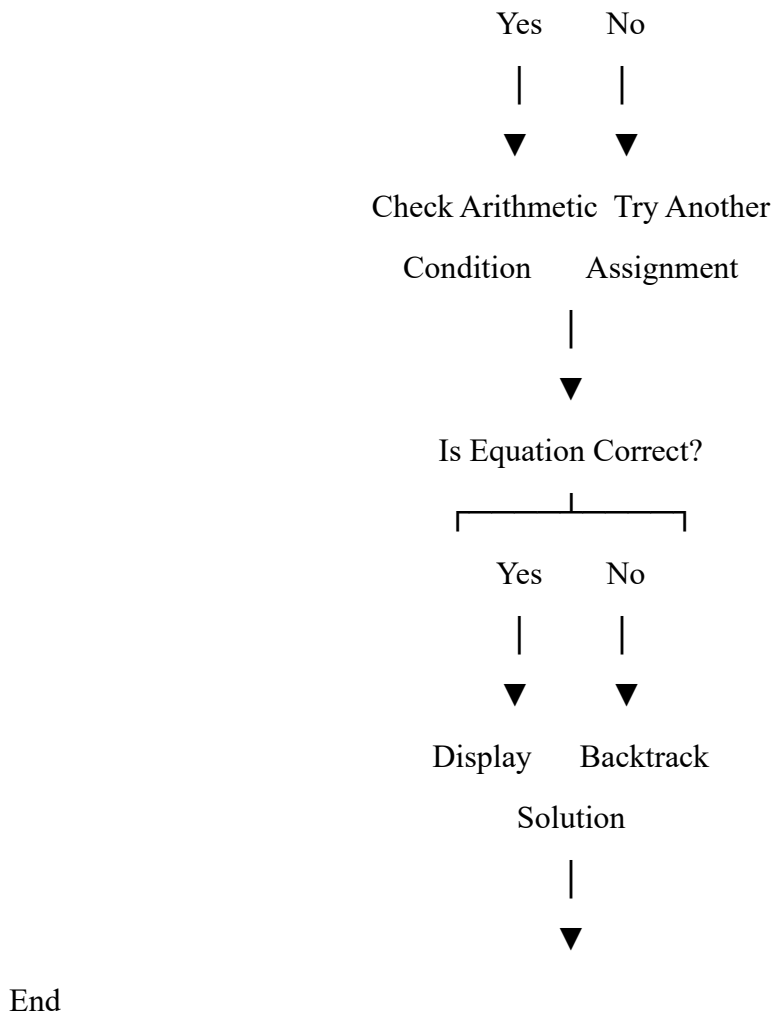
To solve a Cryptarithmic problem using Python by assigning unique digits to letters so that the arithmetic equation is satisfied.

Objective

- To assign unique digits (0–9) to letters in an arithmetic puzzle.
- Ensure that no two letters have the same digit.
- Find a valid assignment that satisfies the arithmetic equation using **Backtracking**.

Flowchart:





Algorithm

1. Import the itertools module.
2. Generate all possible digit assignments for the letters.
3. Ensure each letter has a unique digit.
4. Check that leading letters are not assigned zero.
5. Verify whether the arithmetic equation is satisfied.
6. If a valid solution is found, display it and stop.

Pseudo Code

START

Import permutations from itertools

FOR each possible digit assignment

```

Assign digits to letters
IF leading digit is 0
    Skip
ENDIF
IF SEND + MORE = MONEY
    Print solution
    STOP
ENDIF
ENDFOR
END

```

Python Code

```

import itertools

def solve_cryptarithmic():
    letters = 'S E N D M O R Y'.split()

    for perm in itertools.permutations('0123456789', len(letters)):
        mapping = dict(zip(letters, perm))

        if mapping['S'] == '0' or mapping['M'] == '0':
            continue

        SEND = int(mapping['S'] + mapping['E'] + mapping['N'] + mapping['D'])
        MORE = int(mapping['M'] + mapping['O'] + mapping['R'] + mapping['E'])
        MONEY = int(mapping['M'] + mapping['O'] + mapping['N'] + mapping['E'] +
mapping['Y'])

        if SEND + MORE == MONEY:
            return mapping, SEND, MORE, MONEY

    return None

res = solve_cryptarithmic()

if res:
    mapping, SEND, MORE, MONEY = res
    print("Solution found:")

```

```

for k in sorted(mapping):
    print(f" {k} = {mapping[k]}")

print()

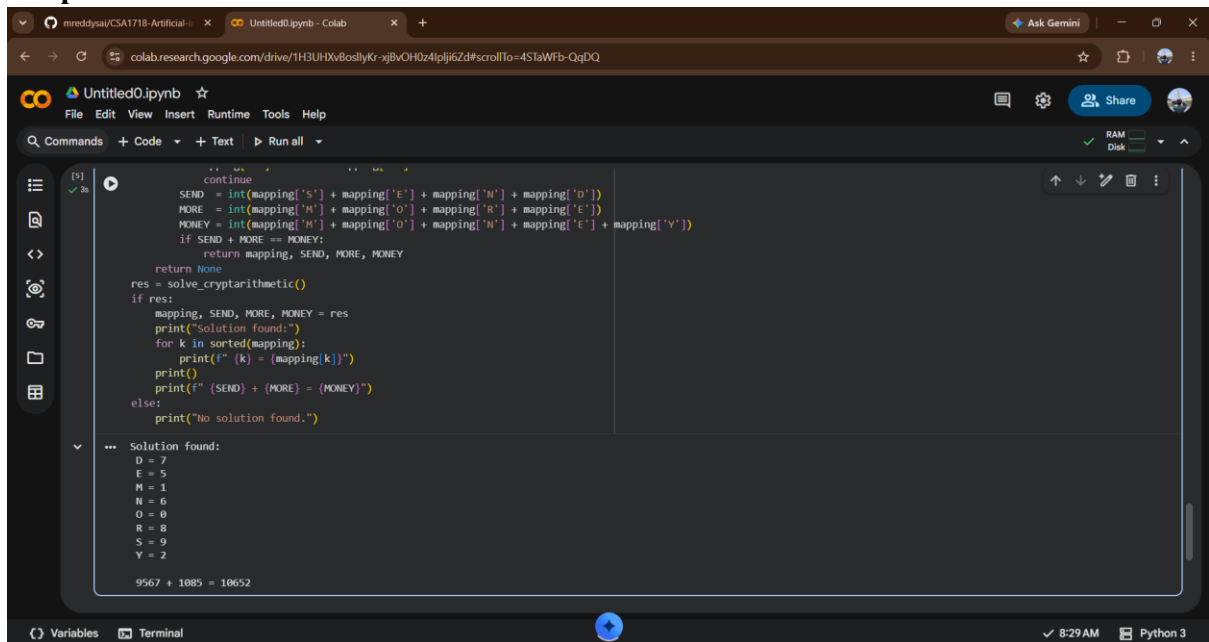
print(f" {SEND} + {MORE} = {MONEY}")

else:

    print("No solution found.")

```

Output



The screenshot shows a Google Colab notebook with a Python script that solves the cryptarithmic problem SEND + MORE = MONEY. The script uses a brute-force approach, iterating through all possible mappings of letters to digits (0-9). It finds a solution where D=7, E=5, N=1, M=6, O=0, R=8, S=9, and Y=2. The output of the script is displayed in the output window, showing the solution found and the equation 9567 + 1085 = 10652.

```

...
continue

SEND = int(mapping['S'] + mapping['E'] + mapping['N'] + mapping['D'])
MORE = int(mapping['M'] + mapping['O'] + mapping['R'] + mapping['E'])
MONEY = int(mapping['M'] + mapping['O'] + mapping['N'] + mapping['E'] + mapping['Y'])
if SEND + MORE == MONEY:
    return mapping, SEND, MORE, MONEY
return None

res = solve_cryptarithmic()
if res:
    mapping, SEND, MORE, MONEY = res
    print("Solution found:")
    for k in sorted(mapping):
        print(f" {k} = {mapping[k]}")
    print()
    print(f" {SEND} + {MORE} = {MONEY}")
else:
    print("No solution found.")

```

... Solution found:
D = 7
E = 5
N = 1
M = 6
O = 0
R = 8
S = 9
Y = 2
9567 + 1085 = 10652

Result

The Cryptarithmic problem **SEND + MORE = MONEY** was successfully solved using Python. Each letter was assigned a unique digit, and the equation was satisfied correctly.